

Než začneš testovat a ladit svůj řešič, zopakuj si, jak se dělá zkouška rovnice, jak Python zachází se vstupem, jak funguje větvení, proč jsou důležité hranice intervalů a jak ručně sledovat proměnné v programu.

1. Zkouška řešení rovnice **ZŠ**

Kde to najdeš: Začátečníci H17 Lineární rovnice I, 1.+2. H46 Odmocnina a zlomek v rovnici

Zkouška ověří, že číslo opravdu je řešením: dosadíš ho do původní rovnice, spočítáš zvlášť levou a pravou stranu a porovnáš je. Když se strany rovnají, je číslo kořenem; jinak ho vyřadíš. U rovnic s odmocninou nebo s neznámou ve jmenovateli je zkouška nutná, protože některé úpravy přidávají falešné kořeny. V programování odpovídá zkoušce test: předem víš, co má vyjít, a porovnáš to se skutečným výsledkem.

Příklad: Jsou čísla 2 a -1 řešením rovnice $\sqrt{x+2} = x$?

1. $x = 2$: $L = \sqrt{4} = 2$, $P = 2 \rightarrow$ kořen.
2. $x = -1$: $L = \sqrt{1} = 1$, $P = -1 \rightarrow$ není kořen.
3. $K = \{2\}$.

Zkus sám:

1. Je $x = 3$ řešením rovnice $x^2 - 2x = 3$?
2. Je $x = -2$ řešením rovnice $\frac{4}{x+2} = 1$?

2. Vstup a převod typů v Pythonu **SŠ**

Kde to najdeš: 1.+2. H41 První program

Funkce `input()` vrací vždy text (typ `str`), i když uživatel napíše číslo. Pro výpočty ho převedeš funkcí `int()` na celé číslo, nebo `float()` na desetinné. Když text číslo neobsahuje, převod skončí chybou `ValueError`. Pozor i na desetinnou čárku: `float('2,5')` selže, Python chce tečku '2.5'. Právě takové vstupy patří do testovací sady.

Příklad: Co udělají příkazy `int('7')`, `float('3.5')`, `int('3.5')` a `float('abc')`?

1. `int('7')` vrátí celé číslo 7.
2. `float('3.5')` vrátí desetinné číslo 3.5.
3. `int('3.5')` skončí chybou `ValueError` – text není zápis celého čísla.
4. `float('abc')` skončí chybou `ValueError` – text není číslo.

Zkus sám:

1. Co vypíše `print(input() * 2)`, když uživatel zadá 5?
2. Jak převedeš vstup '2,5' (s desetinnou čárkou) na číslo?

3. Větvení a složené podmínky **SŠ**

Kde to najdeš: 1.+2. H44 Větvení: řešič rovnic

Příkaz `if` vybere větev podle podmínky; `elif` a `else` doplní další možnosti a provede se vždy jen první splněná větev. Složené podmínky spojuje slovy `and`, `or` a `not`. Každá větev je jiná cesta, kterou mohou data programem projít – a každou cestu je potřeba aspoň jednou otestovat. Řešič kvadratické rovnice má šest větví, a proto potřebuje aspoň šest testů.

Příklad: Program: `if D > 0: print(2), elif D == 0: print(1), else: print(0)`. Jakými třemi hodnotami D ho otestuješ celý?

1. Větev $D > 0$ otestuje třeba $D = 5$ (očekávám 2).
2. Větev $D == 0$ otestuje jen přesně $D = 0$ (očekávám 1).
3. Větev `else` otestuje třeba $D = -3$ (očekávám 0).
4. Tři testy, každý projde jinou větví.

Zkus sám:

1. Kolik testů potřebuješ, aby každá větev programu s `if`, třemi `elif` a jedním `else` proběhla aspoň jednou?
2. Co vypíše `print(3 > 2 and not 5 < 4)`?

4. Hranice intervalu: ostrá a neostrá nerovnost SS

Kde to najdeš: 1.+2. H35 Intervaly, absolutní hodnota

Mnoho chyb vzniká na hranici: programátor napíše $<$ místo $\leq$ a program pak chybuje právě u krajního čísla. Proto se testuje číslo na hranici a čísla těsně pod ní a nad ní. Má-li podmínka platit „od 15 do 17 let včetně“, píšeš $15 \leq \text{vek} \leq 17$ a testuješ 14, 15, 17 a 18. Číslo uprostřed intervalu (16) chybu na hranici neodhalí.

$15 \leq v \leq 17 \Leftrightarrow v \in \langle 15; 17 \rangle$ neostré nerovnosti hranice zahrnují

Příklad: Sleva platí pro děti do 6 let včetně. Programátor napsal `if vek < 6`. Který test chybu odhalí?

1. Správná podmínka je `vek <= 6`.
2. Pro 5 a 7 dávají obě podmínky stejnou odpověď.
3. Liší se jen pro `vek = 6`: správně je sleva, program ji nedá.
4. Chybu odhalí test přesně na hranici, `vek = 6`.

Zkus sám:

1. Jaké hodnoty otestuješ u podmínky „teplota nad 30 °C“?
2. Zapiš intervalem „věk od 15 do 26 let včetně“.

5. Trasovací tabulka – ruční krokování SS

Kde to najdeš: 1.+2. H6 Zápis algoritmu, 1.+2. H41 První program

Trasovací tabulka zapisuje, jak se po každém řádku programu mění hodnoty proměnných: do sloupců dáš proměnné, do řádků jednotlivé kroky. Zjistíš tak, co program opravdu dělá, a ne co si myslíš, že dělá. Stejnou práci za tebe udělá krokování v Python Tutor nebo v Thonny, ruční tabulka tě ale naučí program číst.

Příklad: Sestav trasovací tabulku pro program `a = 3, b = a * 2, a = a + b, print(a, b)`.

1. Po 1. řádku: `a = 3`, `b` zatím neexistuje.
2. Po 2. řádku: `a = 3`, `b = 6`.
3. Po 3. řádku: `a = 9`, `b = 6`.
4. Program vypíše 9 6.

Zkus sám:

1. Co vypíše program `x = 5, y = x, x = x - 2, print(x, y)`?
2. Co vypíše program `a = 2, a = a * a, a = a * a, print(a)`?

Kde se to doučit

- **Výjimky – Nauč se Python** – Běhové chyby jako výjimky, jejich druhy a zachycení pomocí `try/except`.
<https://naucse.python.cz/course/pyladies/beginners/exceptions/>
- **Testování – Nauč se Python** – Automatické testy s `assert` a `pytest`, testovací případy a hraniční hodnoty.
<https://naucse.python.cz/course/pyladies/beginners/testing/>
- **Python Tutor – krokování programu** – Program spustíš po jednotlivých krocích a sleduješ hodnoty proměnných; ideální k hledání logických chyb.
<https://pythontutor.com/visualize.html>

Výsledky ke kontrole

- **1. Zkouška řešení rovnice:** 1) ano: $9 - 6 = 3$; 2) není – pro $x = -2$ je jmenovatel nulový
- **2. Vstup a převod typů v Pythonu:** 1) 55 – text se zopakuje dvakrát; 2) `float('2,5'.replace(',', '.'))` dá 2.5
- **3. Větvení a složené podmínky:** 1) 5; 2) True
- **4. Hranice intervalu: ostrá a neostrá nerovnost:** 1) 30 (neplatí), těsně nad 30, např. 30,1 (platí), a něco pod hranicí, např. 29; 2) $\langle 15; 26 \rangle$
- **5. Trasovací tabulka – ruční krokování:** 1) 3 5; 2) 16